

ECE 2011

Introduction to Computer Engineering
Spring 2026

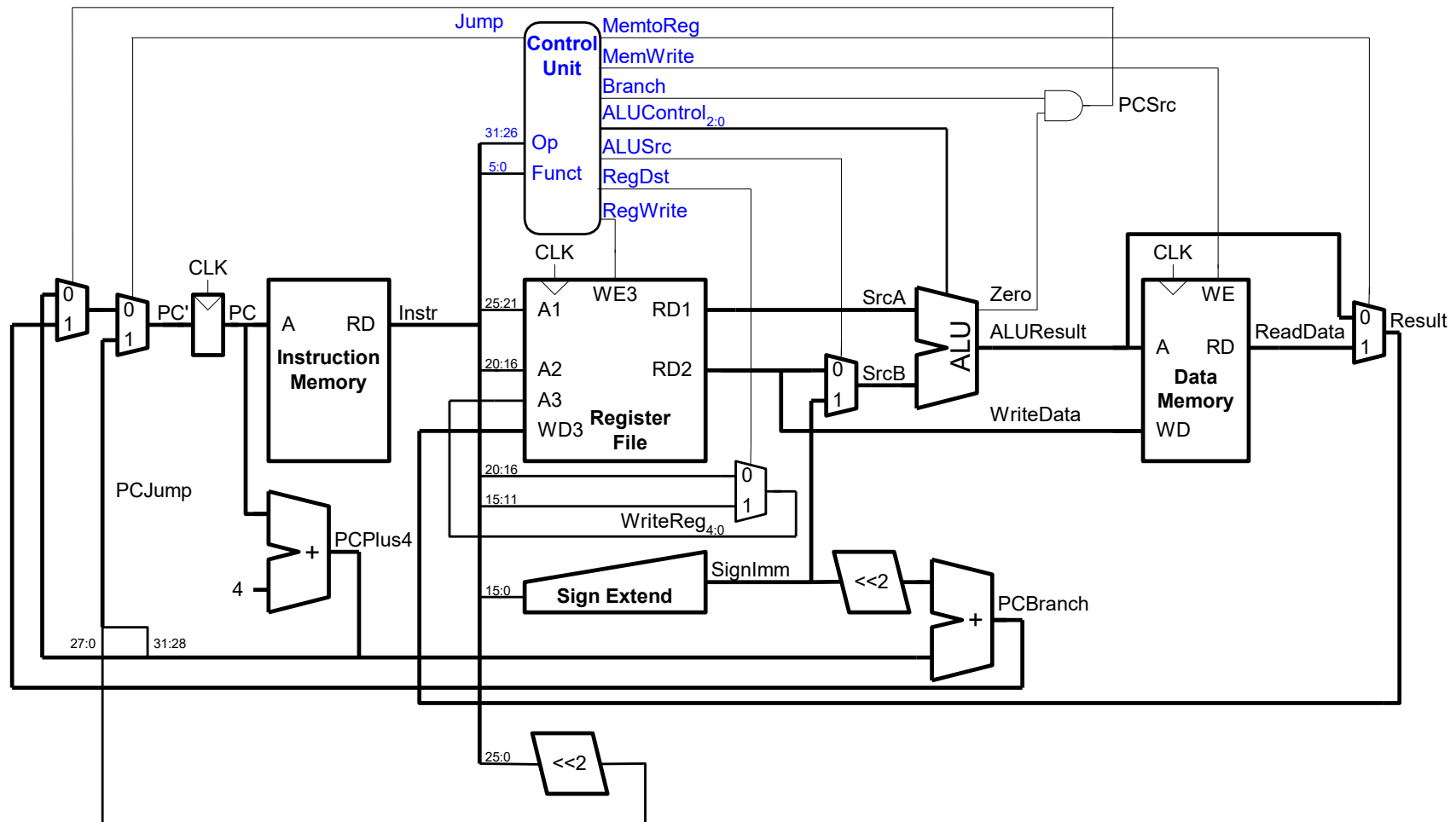
Lecture 16

03-26-2026

Linghao Song

Yale ECE

Single-Cycle Processor & Lab7



Operands

- Operand location: physical location in computer
 - Registers
 - Memory
 - Constants (also called *immediates*)

Operands: Registers

- MIPS has 32 32-bit registers
- Registers are faster than memory
- MIPS called “32-bit architecture” because it operates on 32-bit data

MIPS Register Set

Name	Register Number	Usage
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

Operands: Registers

- **Registers:**
 - \$ before name
 - Example: \$0, “register zero”, “dollar zero”
- Registers used for **specific purposes**:
 - \$0 always holds the **constant value 0**.
 - the ***saved registers***, \$s0 – \$s7, used to hold variables
 - the ***temporary registers***, \$t0 – \$t9, used to hold intermediate values during a larger computation

Instructions with Registers

- Revisit add instruction

C Code

```
a = b + c
```

MIPS assembly code

```
# $s0 = a, $s1 = b, $s2 = c  
add $s0, $s1, $s2
```

Reading Memory

- Memory read called **load**
- **Mnemonic:** *load word* (lw)
- **Format:**
 $lw \$s0, 5(\$t1)$
- **Address calculation:**
 - add *base address* ($\$t1$) to the *offset* (5)
 - $address = (\$t1 + 5)$
- **Result:**
 - $\$s0$ holds the value at address $(\$t1 + 5)$

Any register may be used as base address

Writing Memory

- Memory write are called **store**
- **Mnemonic:** *store word* (sw)
- **Example:** Write (store) the value in $\$t4$ into memory address 7
 - add the base address ($\$0$) to the offset (0x7)
 - address: $(\$0 + 0x7) = 7$Offset can be written in decimal (default) or hexadecimal

Assembly code

```
sw $t4, 0x7($0)    # write the value in $t4  
                   # to memory word 7
```

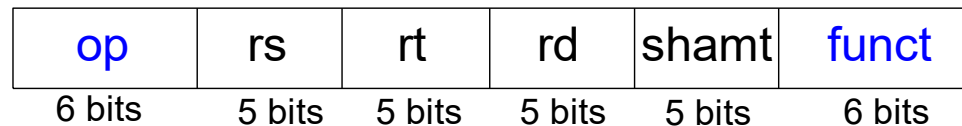
Machine Language

- Binary representation of instructions
- Computers only understand 1's and 0's
- 32-bit instructions
 - Simplicity favors regularity: 32-bit data & instructions
- 3 instruction formats:
 - **R-Type**: register operands
 - **I-Type**: immediate operand
 - **J-Type**: for jumping

R-Type

- *Register-type*
- 3 register operands:
 - rs, rt: source registers
 - rd: destination register
- Other fields:
 - op: the *operation code* or *opcode* (0 for R-type instructions)
 - funct: the *function*
with opcode, tells computer what operation to perform
 - shamt: the *shift amount* for shift instructions, otherwise it's 0

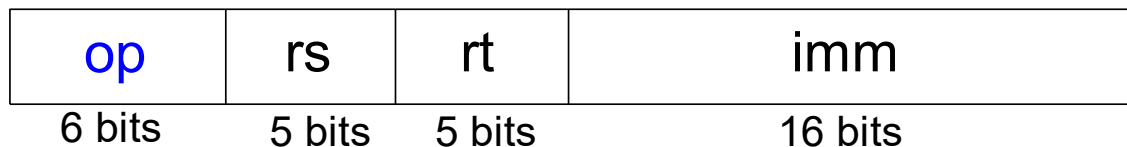
R-Type



I-Type

- *Immediate-type*
- 3 operands:
 - `rs, rt`: register operands
 - `imm`: 16-bit two's complement immediate
- Other fields:
 - `op`: the opcode
 - Simplicity favors regularity: all instructions have opcode
 - Operation is completely determined by opcode

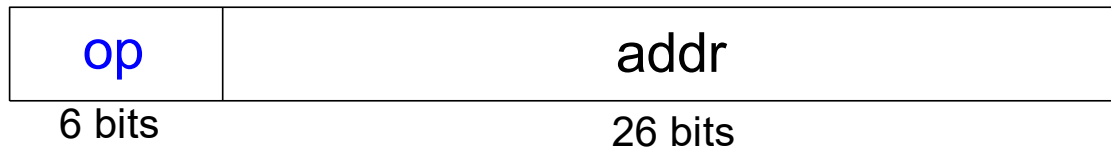
I-Type



Machine Language: J-Type

- *Jump-type*
- 26-bit address operand (`addr`)
- Used for jump instructions (`j`)

J-Type



Review: Instruction Formats

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

J-Type

op	addr
6 bits	26 bits

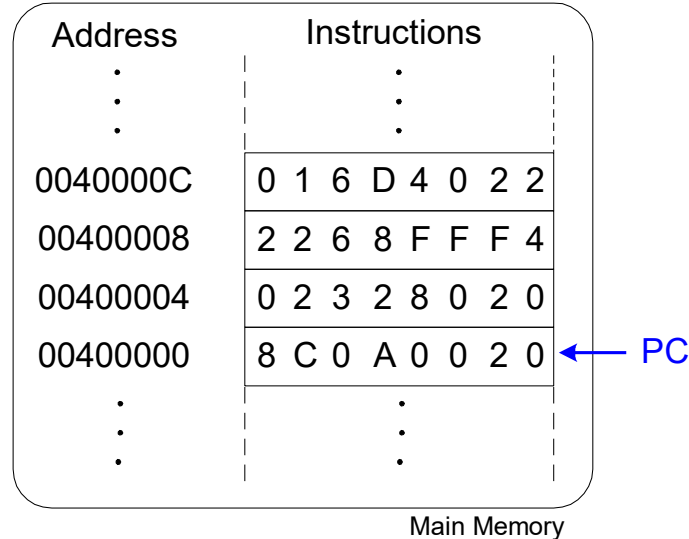
Power of the Stored Program

- 32-bit instructions & data stored in memory
- Sequence of instructions: only difference between two applications
- To run a new program:
 - No rewiring required
 - Simply store new program in memory
- Program Execution:
 - Processor *fetches* (reads) instructions from memory in sequence
 - Processor performs the specified operation

The Stored Program

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022

Stored Program



Program Counter (PC): keeps track of current instruction

Branching

- Execute instructions out of sequence
- Types of branches:
 - **Conditional**
 - branch if equal (`beq`)
 - branch if not equal (`bne`)
 - **Unconditional**
 - jump (`j`)
 - jump register (`j r`)
 - jump and link (`j al`)

Conditional Branching (beq)

MIPS assembly

```
addi $s0, $0, 4      # $s0 = 0 + 4 = 4
addi $s1, $0, 1      # $s1 = 0 + 1 = 1
sll  $s1, $s1, 2      # $s1 = 1 << 2 = 4
beq  $s0, $s1, target # branch is taken
addi $s1, $s1, 1      # not executed
sub  $s1, $s1, $s0     # not executed

target:               # label
add  $s1, $s1, $s0     # $s1 = 4 + 4 = 8
```

Labels indicate instruction location. They can't be reserved words and must be followed by colon (:)

Unconditional Branching (j)

MIPS assembly

```
addi $s0, $0, 4           # $s0 = 4
addi $s1, $0, 1           # $s1 = 1
j      target             # jump to target
sra    $s1, $s1, 2         # not executed
addi   $s1, $s1, 1         # not executed
sub    $s1, $s1, $s0       # not executed

target:
add    $s1, $s1, $s0       # $s1 = 1 + 4 = 5
```

Arrays using For Loops

// C Code

```
int array[1000];  
int i;  
  
for (i=0; i < 1000; i = i + 1)  
    array[i] = array[i] * 8;
```

MIPS assembly code

```
# $s0 = array base address, $s1 = i
```

Arrays Using For Loops

MIPS assembly code

\$s0 = array base address, \$s1 = i

initialization code

lui \$s0, 0x23B8 # \$s0 = 0x23B80000

ori \$s0, \$s0, 0xF000 # \$s0 = 0x23B8F000

addi \$s1, \$0, 0 # i = 0

addi \$t2, \$0, 1000 # \$t2 = 1000

loop:

slt \$t0, \$s1, \$t2 # i < 1000?

beq \$t0, \$0, done # if not then done

sll \$t0, \$s1, 2 # \$t0 = i * 4 (byte offset)

add \$t0, \$t0, \$s0 # address of array[i]

lw \$t1, 0(\$t0) # \$t1 = array[i]

sll \$t1, \$t1, 3 # \$t1 = array[i] * 8

sw \$t1, 0(\$t0) # array[i] = array[i] * 8

addi \$s1, \$s1, 1 # i = i + 1

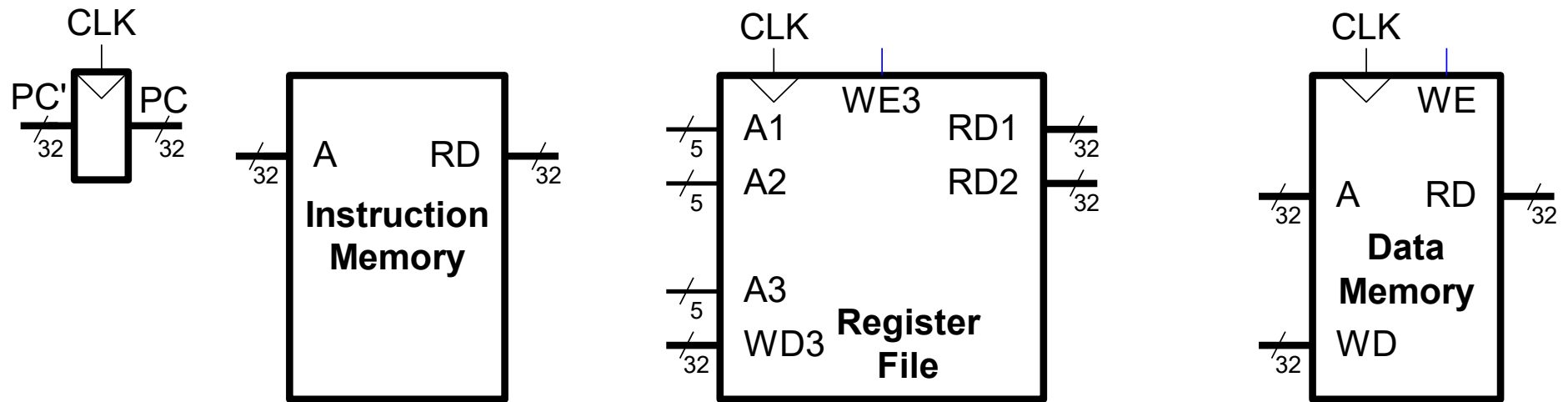
j loop # repeat

done:

Architectural State

- Determines everything about a processor:
 - PC
 - 32 registers
 - Memory

MIPS State Elements

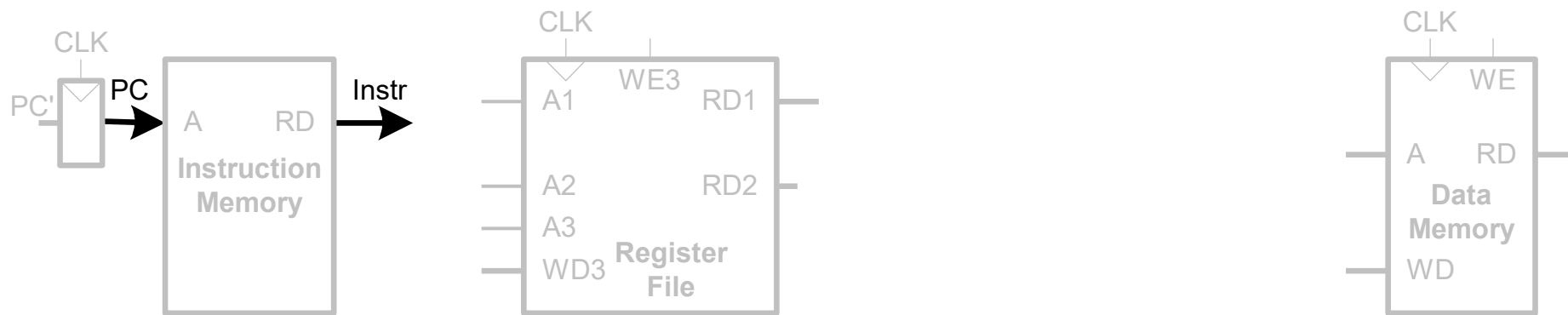


Single-Cycle MIPS Processor

- Datapath
- Control

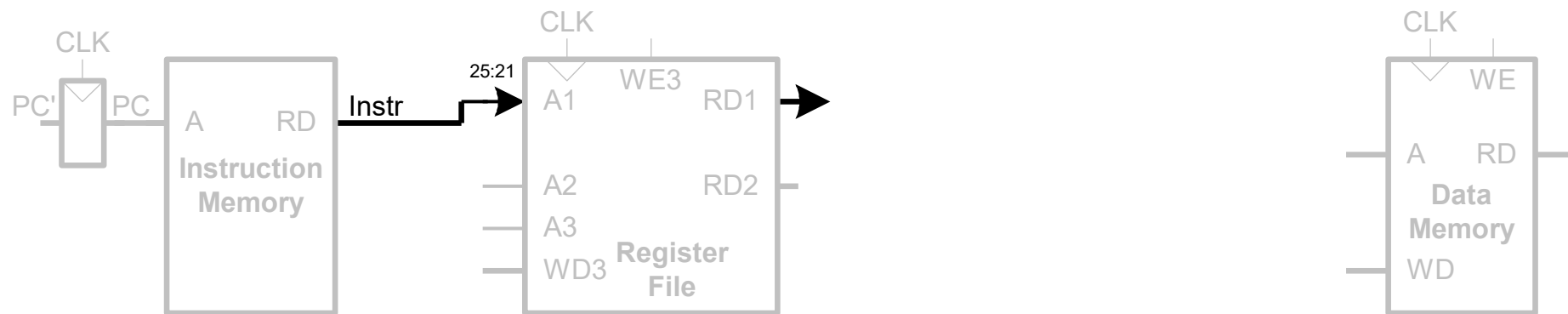
Single-Cycle Datapath: 1w fetch

STEP 1: Fetch instruction



Single-Cycle Datapath: lw Register Read

STEP 2: Read source operands from RF



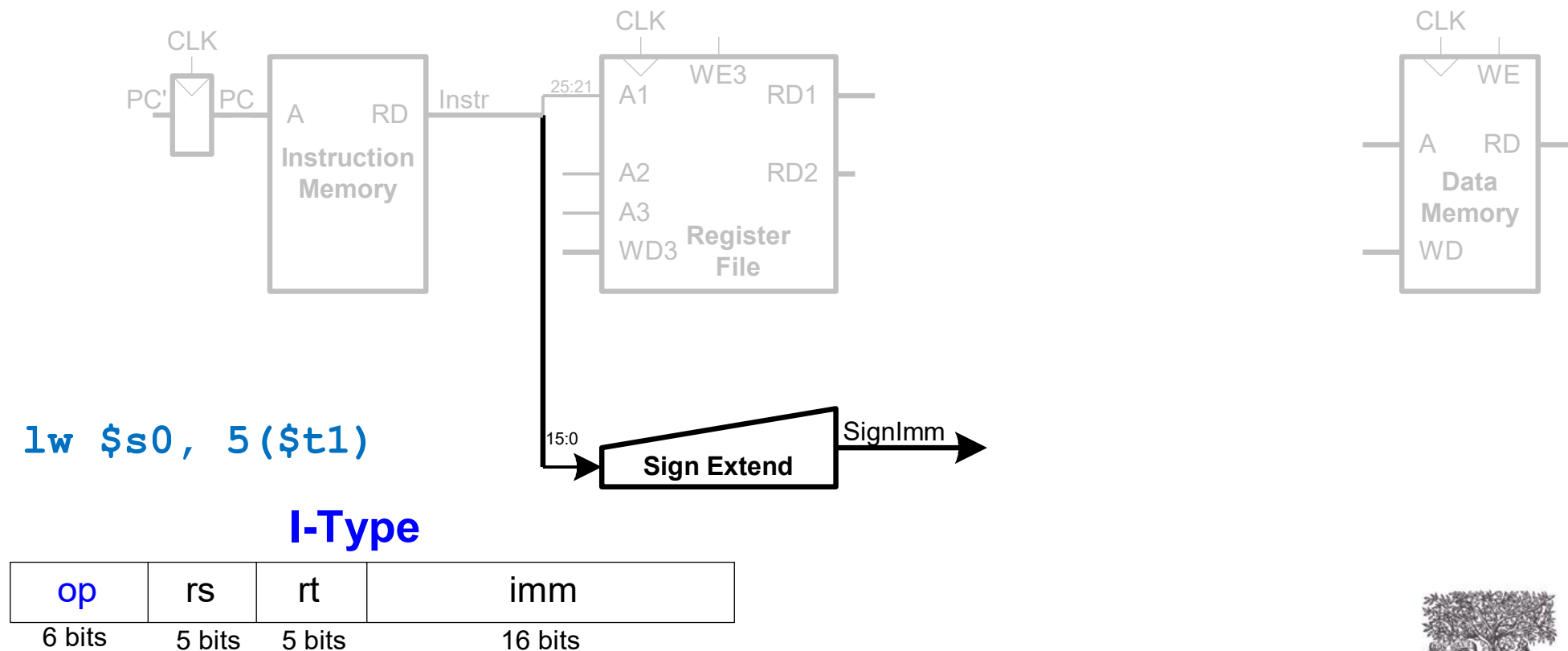
`lw $s0, 5($t1)`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: lw Immediate

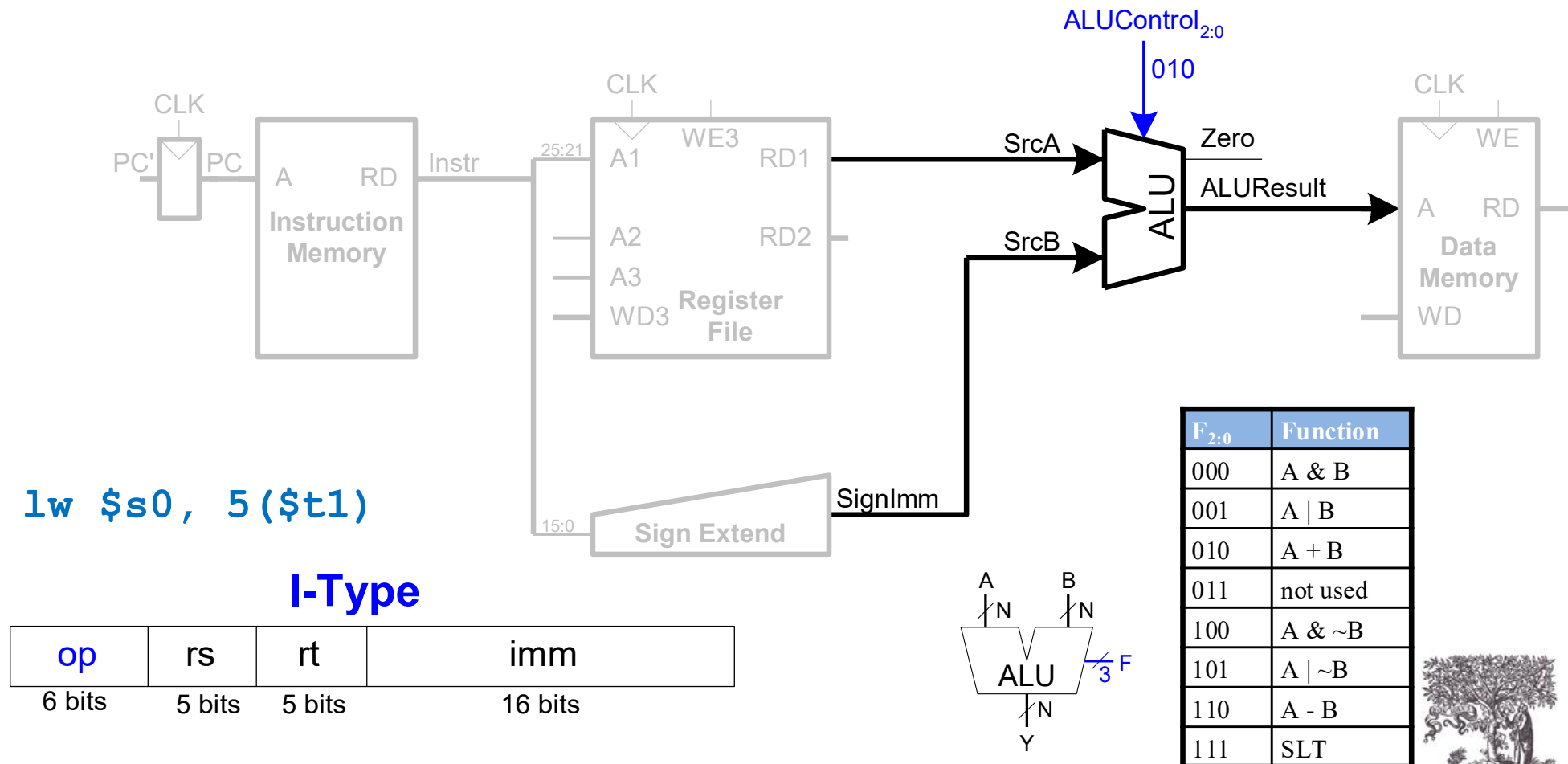
STEP 3: Sign-extend the immediate



I-Type

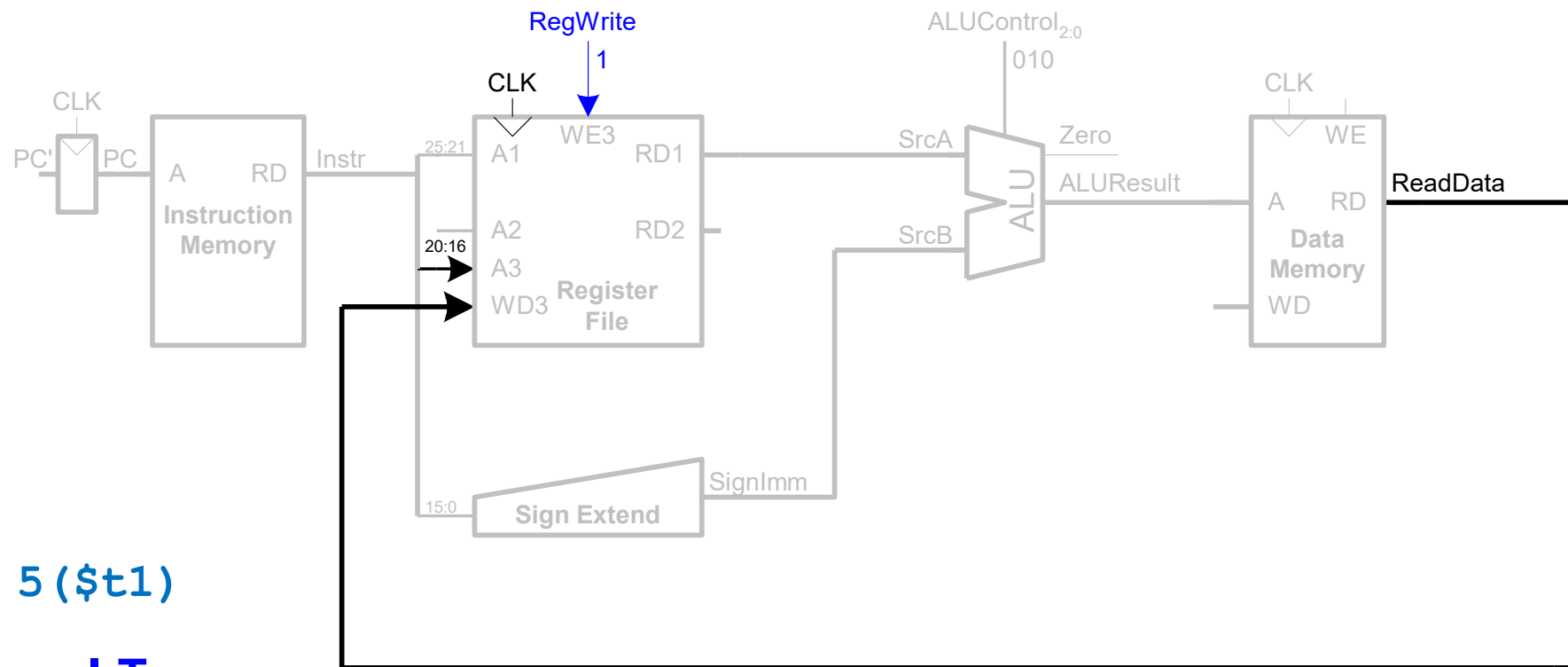
Single-Cycle Datapath: 1w address

STEP 4: Compute the memory address



Single-Cycle Datapath: lw Memory Read

- **STEP 5:** Read data from memory and write it back to register file



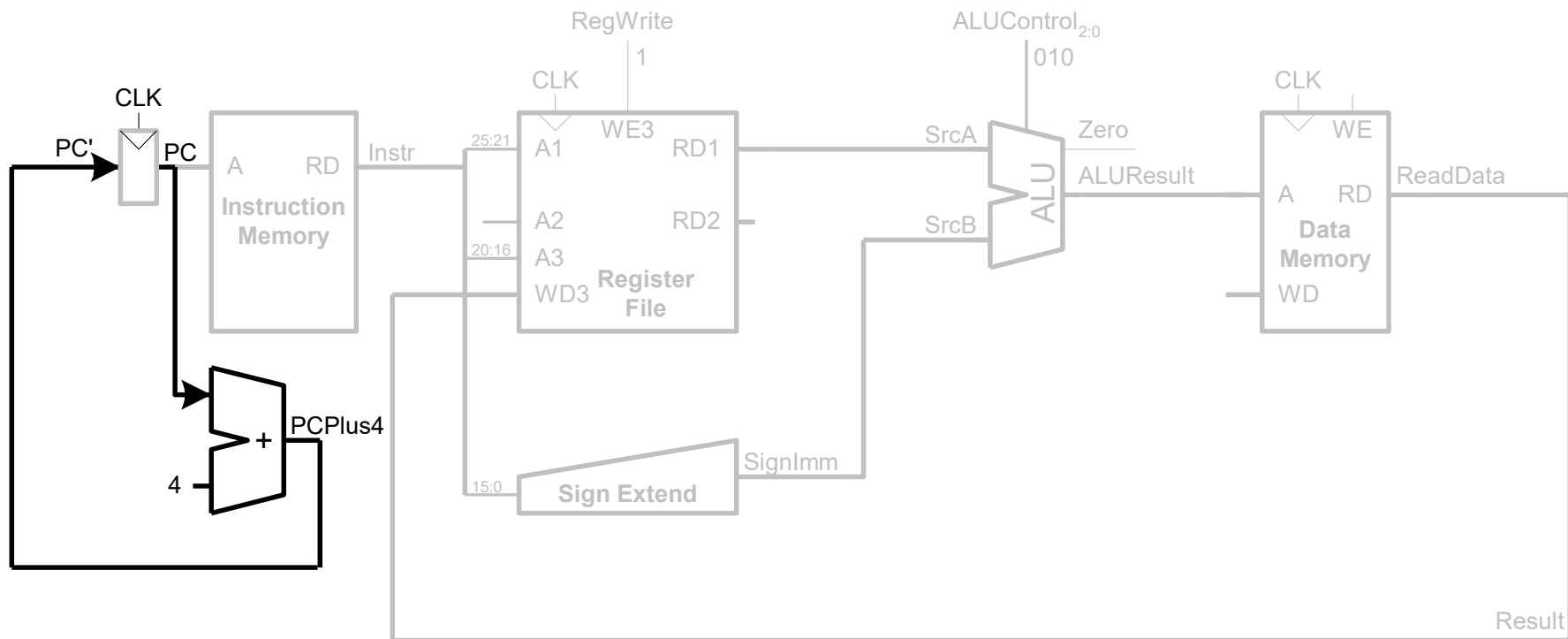
$lw \$s0, 5(\$t1)$

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

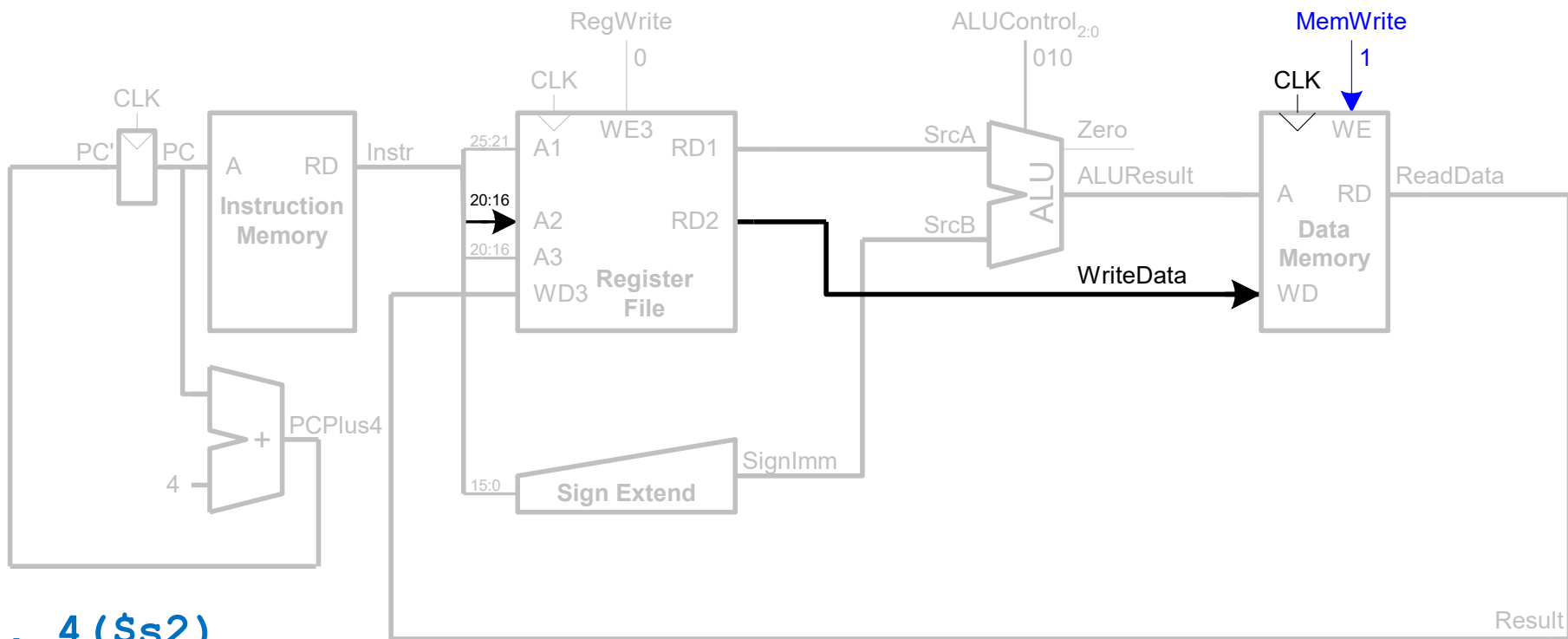
Single-Cycle Datapath: 1_W PC Increment

STEP 6: Determine address of next instruction



Single-Cycle Datapath: sw

Write data in rt to memory



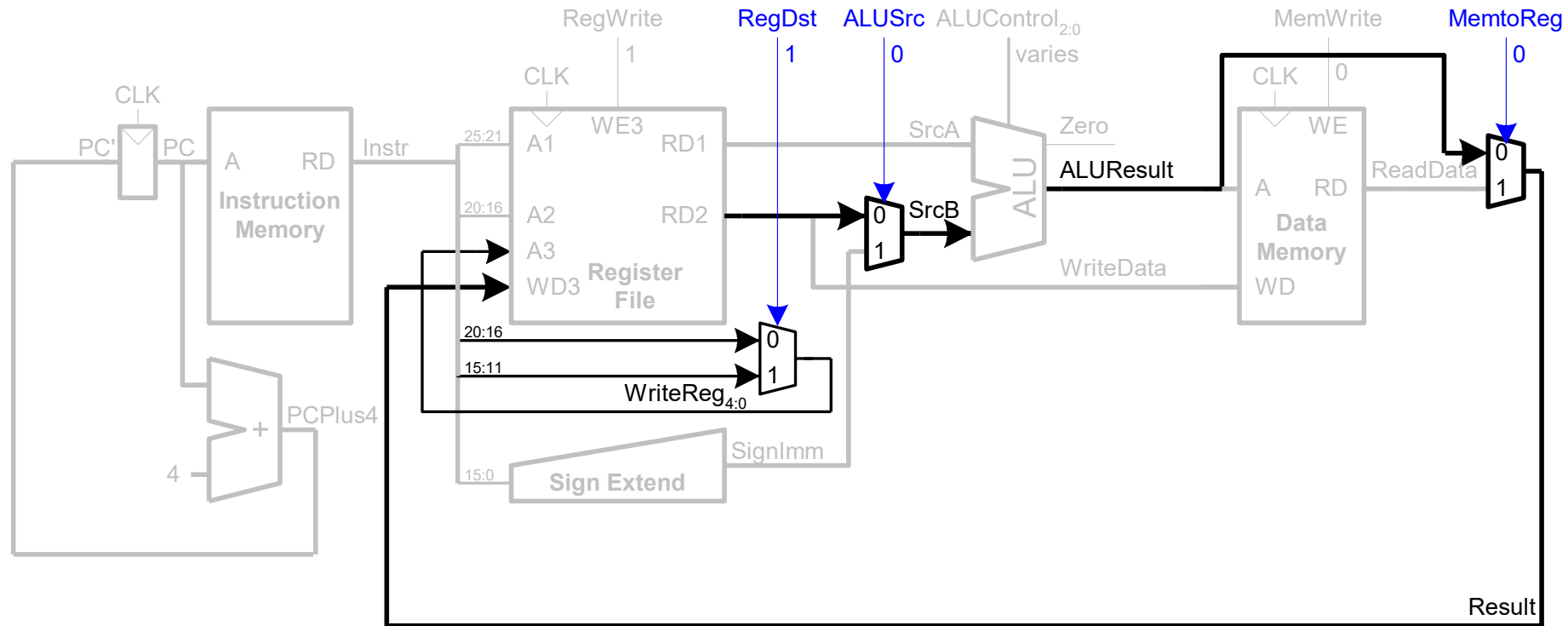
`sw $t1, 4($s2)`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: R-Type

- Read from `rs` and `rt`
- Write *ALUResult* to register file
- Write to `rd` (instead of `rt`)



`add $s2, $s1, $s0`

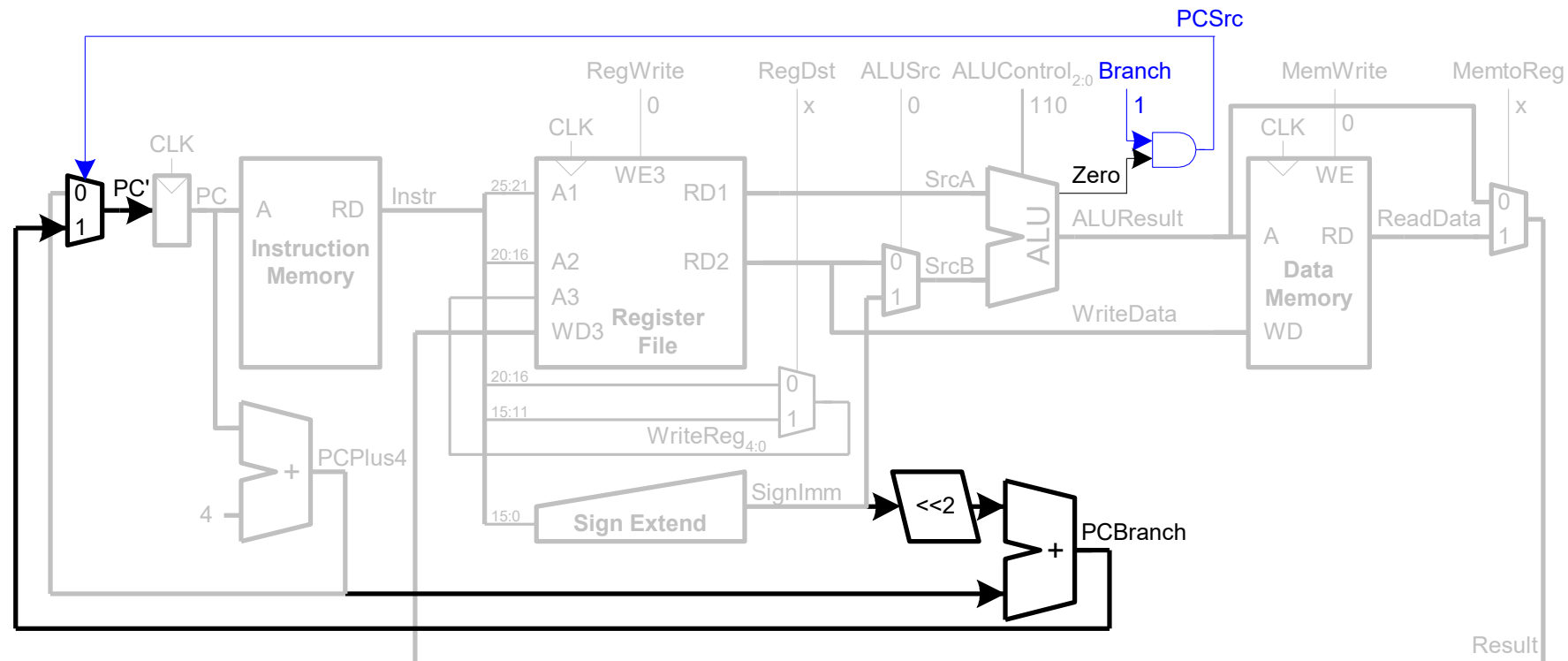
R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Single-Cycle Datapath: beq

- Determine whether values in `rs` and `rt` are equal
- Calculate branch target address:

$$\text{BTA} = (\text{sign-extended immediate} \ll 2) + (\text{PC} + 4)$$

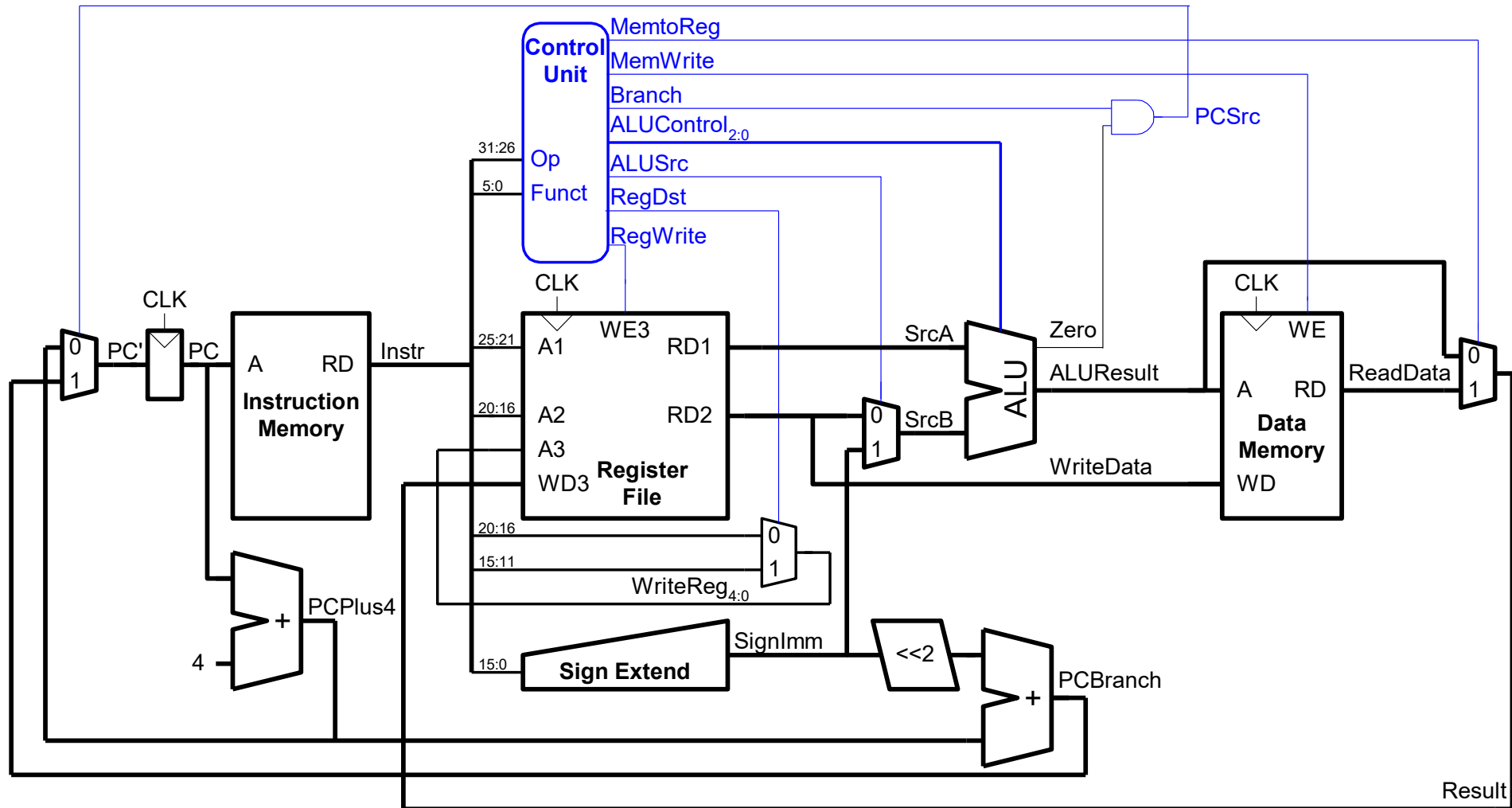


`beq $s0, $s1, target`

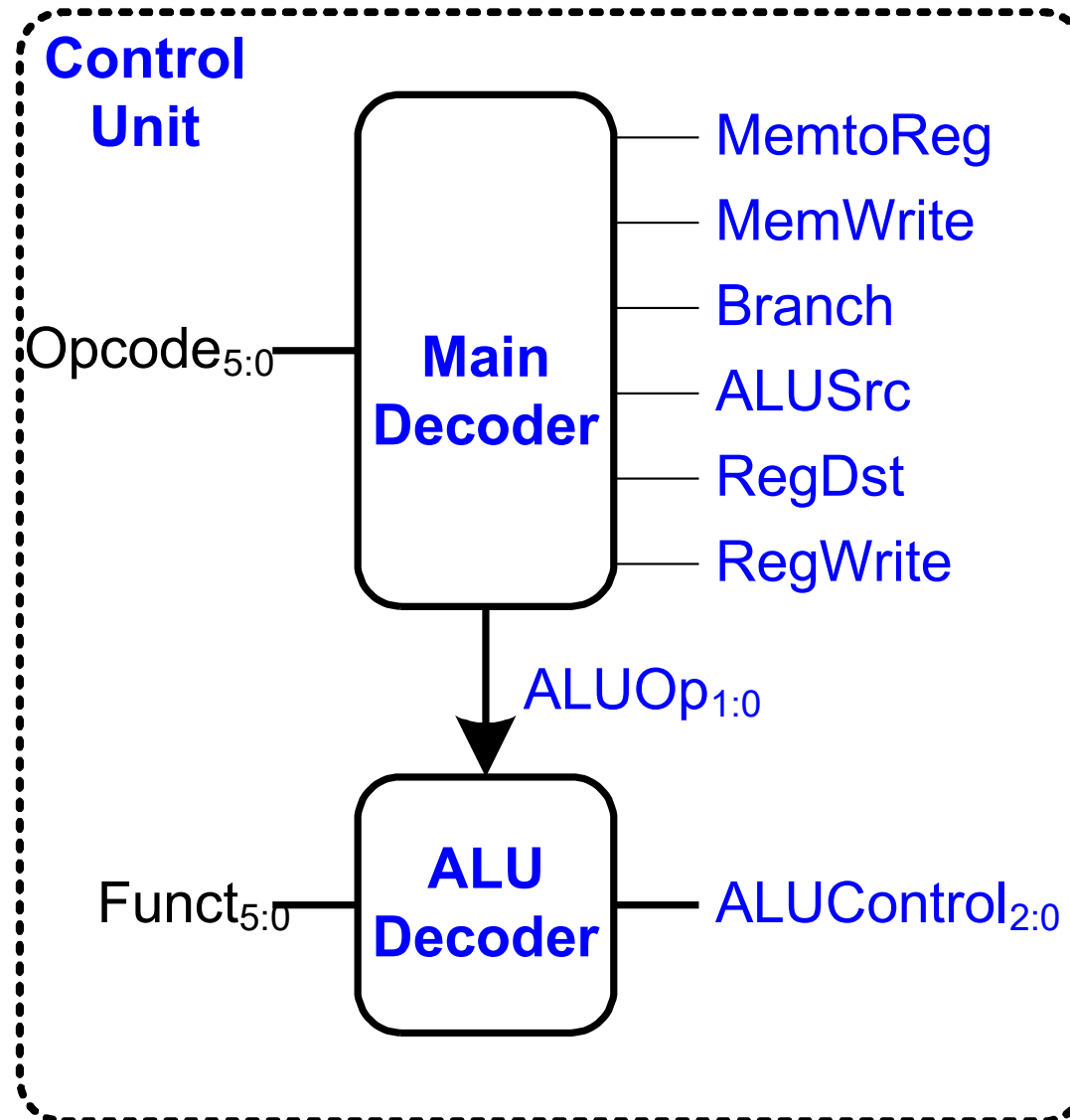
I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

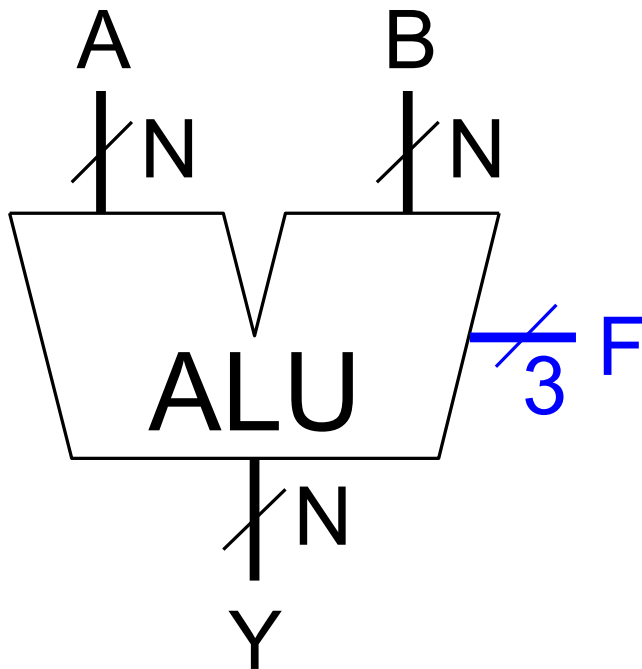
Single-Cycle Processor



Single-Cycle Control

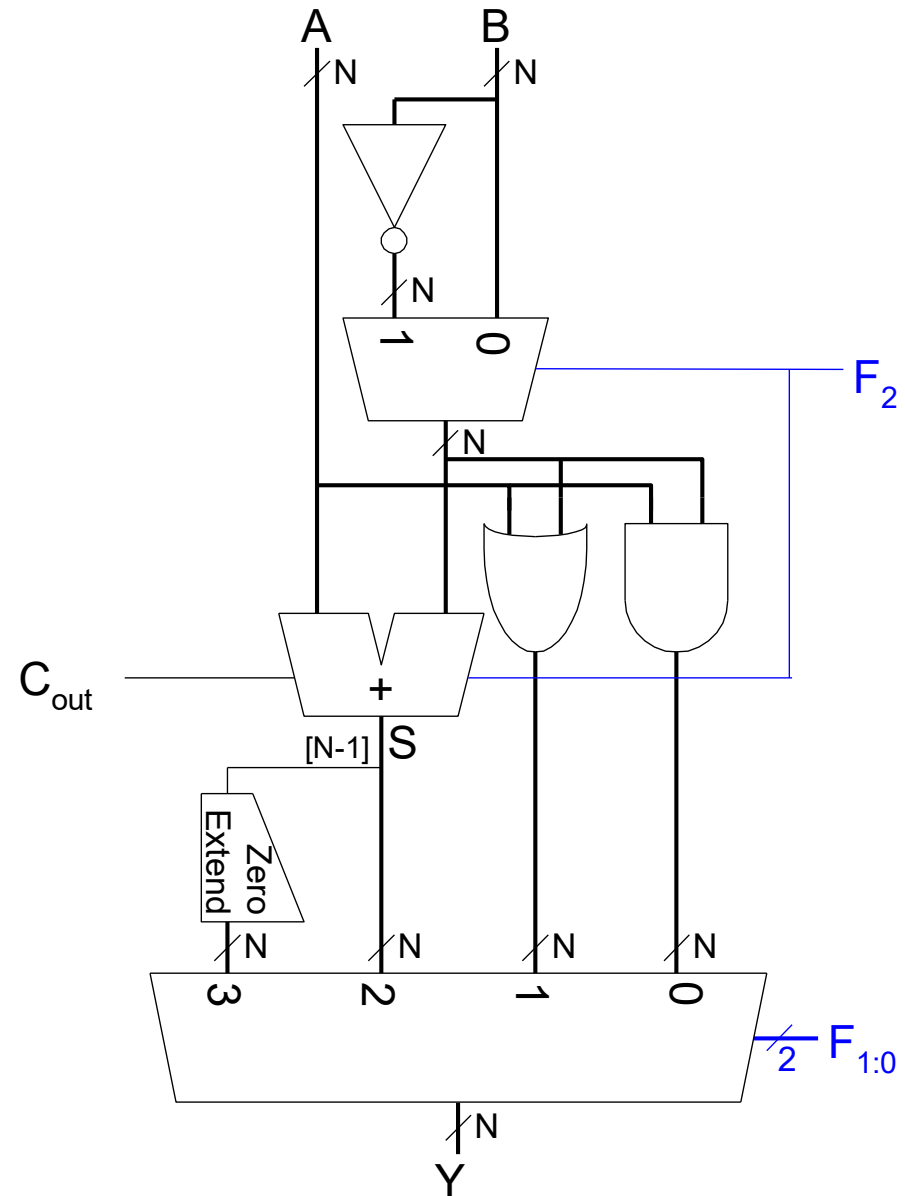


Review: ALU



$F_{2:0}$	Function
000	$A \& B$
001	$A B$
010	$A + B$
011	not used
100	$A \& \sim B$
101	$A \sim B$
110	$A - B$
111	SLT

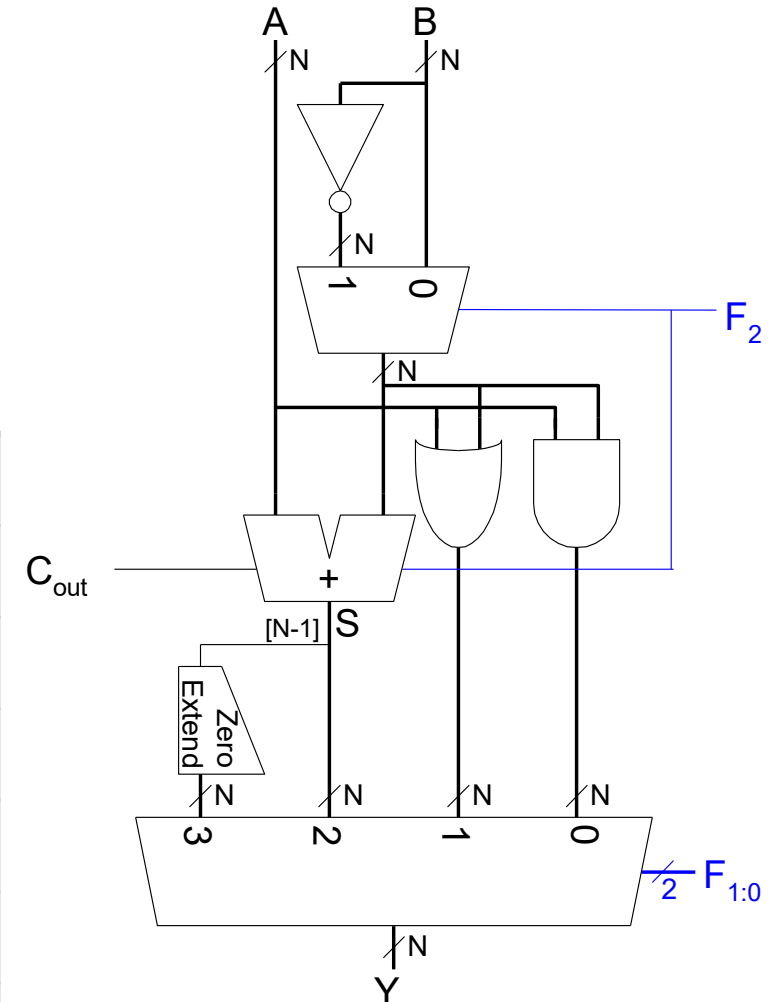
Review: ALU



Control Unit: ALU Decoder

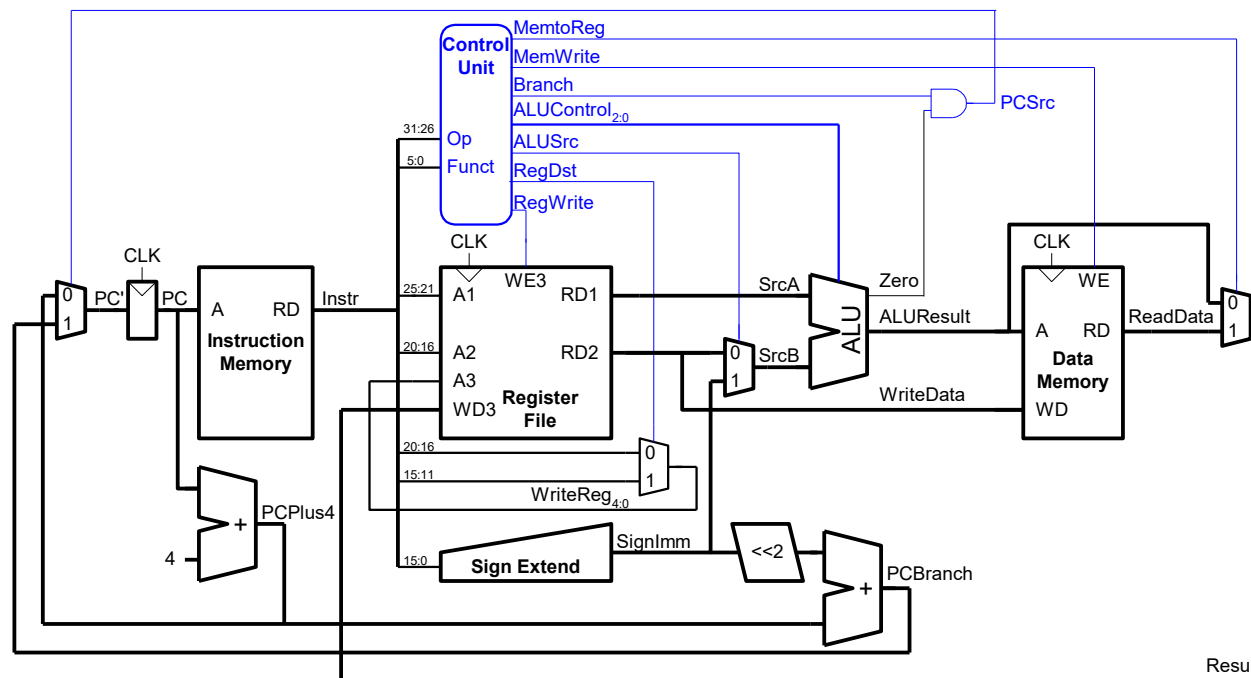
ALUOp _{1:0}	Meaning
00	Add
01	Subtract
10	Look at Funct
11	Not Used

ALUOp _{1:0}	Funct	ALUControl _{2:0}
00	X	010 (Add)
X1	X	110 (Subtract)
1X	100000 (add)	010 (Add)
1X	100010 (sub)	110 (Subtract)
1X	100100 (and)	000 (And)
1X	100101 (or)	001 (Or)
1X	101010 (slt)	111 (SLT)



Control Unit Main Decoder

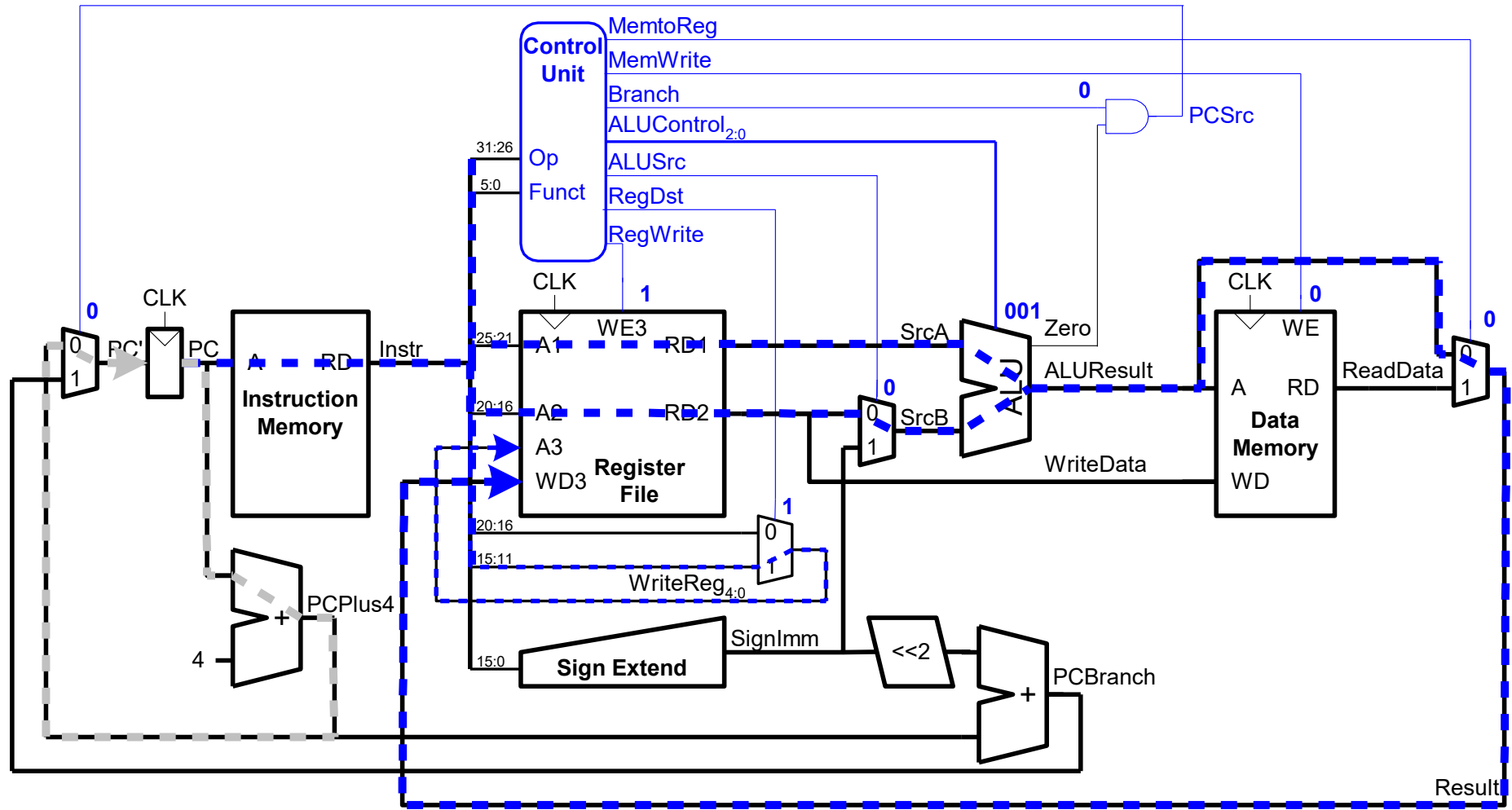
Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000							
lw	100011							
sw	101011							
beq	000100							



Control Unit: Main Decoder

Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	0	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01

Single-Cycle Datapath: or

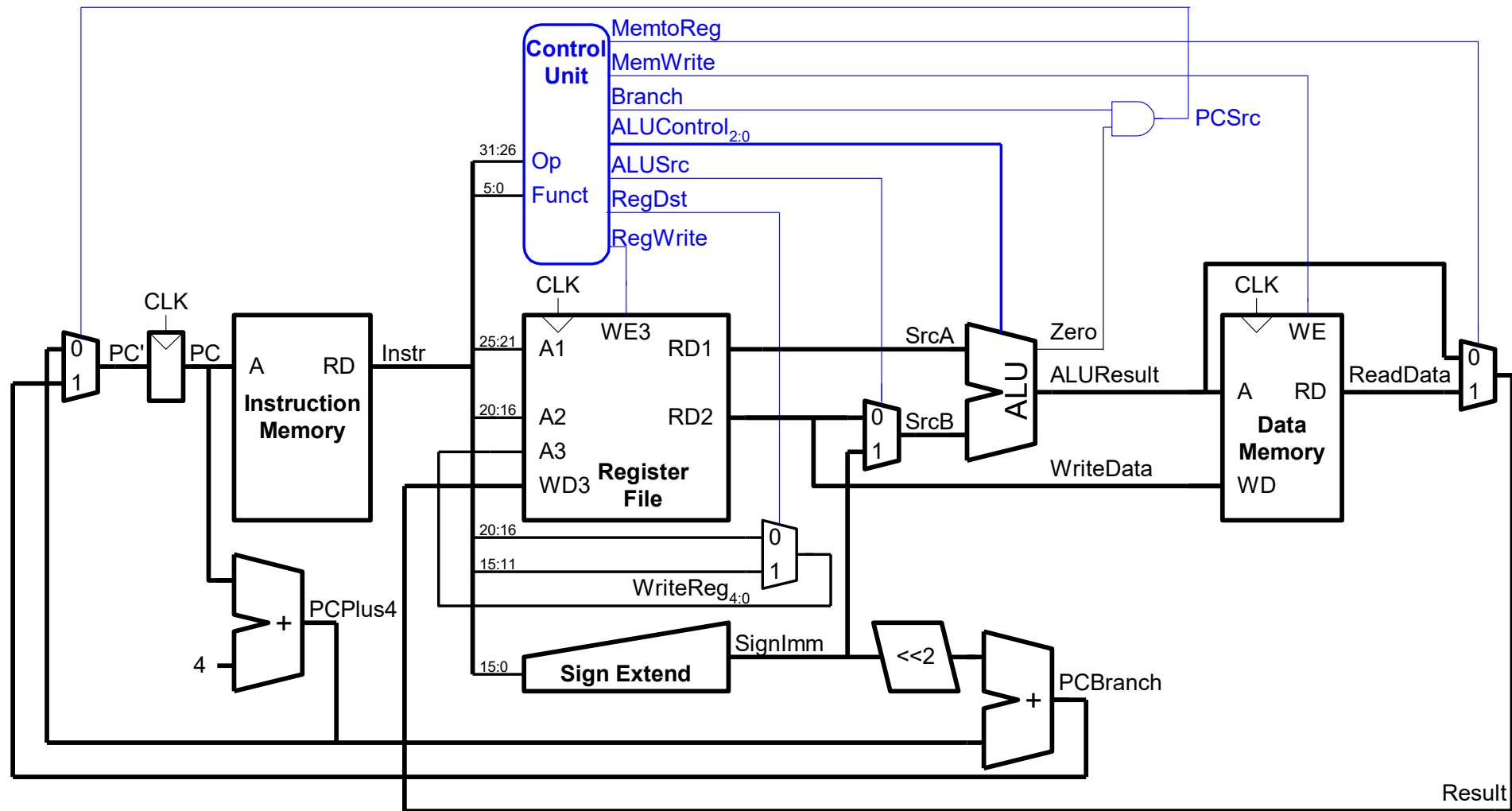


or \$t0, \$t1, \$t2

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Extended Functionality: addi



No change to datapath

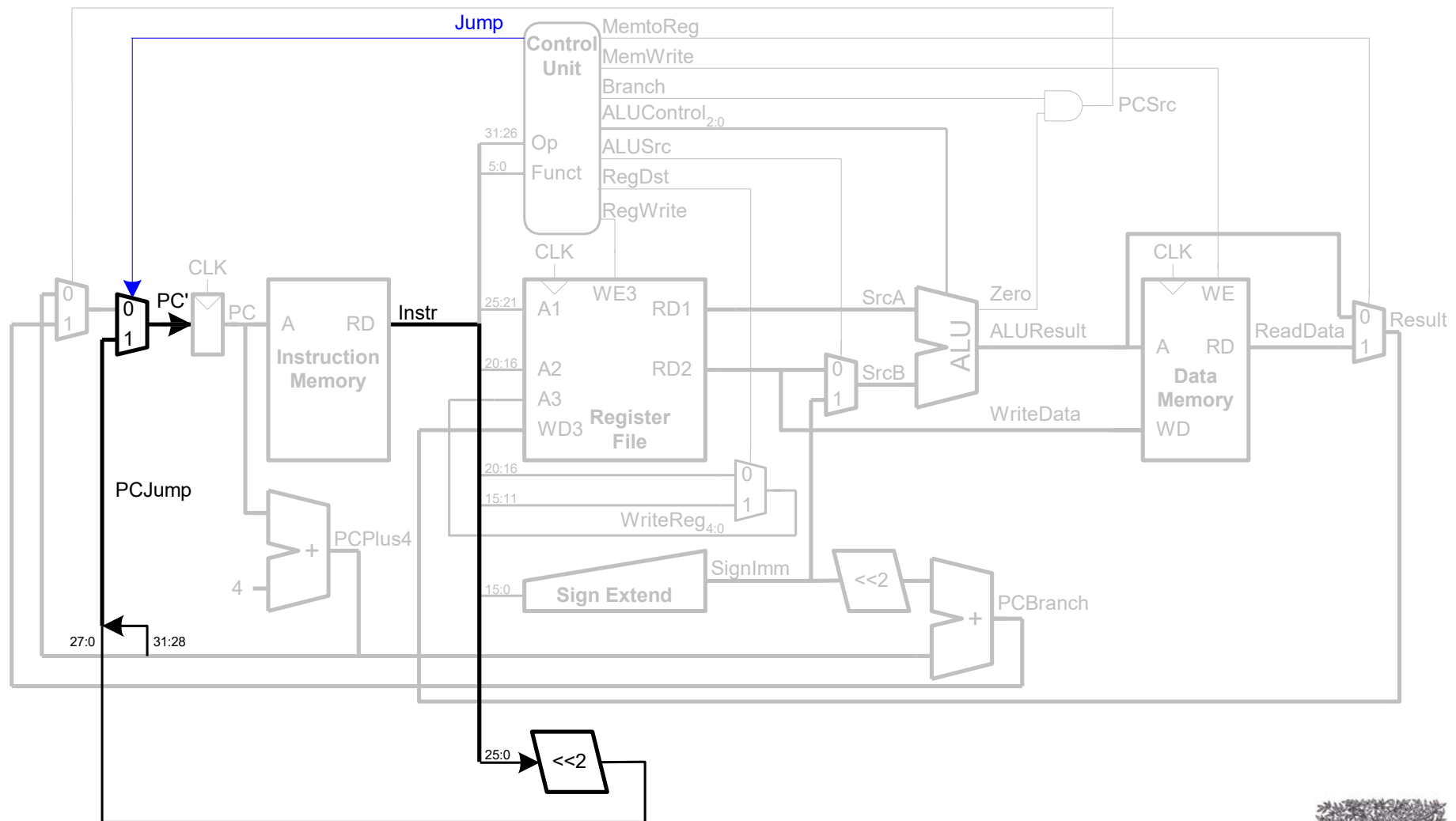
Control Unit: addi

Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000							

Control Unit: addi

Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000	1	0	1	0	0	0	00

Extended Functionality: j



Control Unit: Main Decoder

Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
j	000010								

Control Unit: Main Decoder

Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
j	000010	0	X	X	X	0	X	XX	1